The Bash Scripting Handbook

Table of Contents

Table of Contents 2

Introduction 5

1. Getting Started: the Shebang, echo, read, and Variables 5

addition.sh 6

2. Arithmetic Expansion: $((...)) 6

subtraction.sh 7

multiplication.sh 7

division.sh 7

3. Conditional Statements: if / else and Test Operators 8

evenodd.sh 8

4. while Loops 9

factorial.sh 9

table.sh 9

5. Digit Extraction with while Loops 10

armstrong.sh 10

prime.sh 11

6. case Statements 12

calculator.sh 12

simplecalc.sh 13

7. String Manipulation with tr, and Redirection 14

lowercase.sh 15

convertlowercase.sh 15

8. Command-Line Arguments: $1, $2, and $# 16

dec2hex.sh 16

hextodec.sh 16

rang-random.sh 17

9. C-style for Loops and Arrays 17

decimal2binary.sh 17

10. File Test Operators 18

test-file.sh 19

11. Positional Parameters and for-in Loops 19

list-dir.sh 20

12. Command Substitution and Everyday Utilities 20

count-lines.sh 20

directorysize.sh 21

13. Interactive Prompts and String Concatenation 21
interactive.sh 21
14. Building Paths and Changing Directories in Scripts 22
up.sh 22
15. Custom Functions and Version Sorting 23
versioncompare.sh 23
16. Reading a File Line by Line 24
while-read.sh 24
17. Nested for Loops and Pattern Printing 25
special-pattern.sh 25
18. Infinite Loops, sleep, and Cursor Control 26
Fibonacci.sh 27
color.sh 27
19. Menu-Driven Scripts with Here-Documents 28
read-menu.sh 28
while-menu.sh 29
20. Multi-Level Menus Built from Functions 30
hardware_machine.sh 30
21. Signal Handling with trap 32
traps.sh 32
22. Process Information and Simple File Logging 33
process.sh 33
randomfile.sh 34
23. Talking to the Network with curl 34
weather.sh 34
whereIP.sh 35
24. Batch Image Processing 36
thumbnail.sh 36
25. System Monitoring and Email Alerts 36
cpu.sh 37
disk-space.sh 37
get-temperature.sh 38
26. System Administration and Reporting Scripts 38
server-health.sh 38
collectnetworkinfo.sh 39
27. Archiving and Encryption 41
archive-and-encrypt.sh 41

encrypt.sh 42
28. Remote Backups with rsync 42
remotebackup.sh 42
29. Time-Based Scripts and Cross-Platform Checks 43
pomodoro.sh 43
Closing Notes 44

Introduction

A terminal is simply a window where you type text commands instead of clicking on things, and the shell is the program that sits behind that window, reading each command you type and carrying it out. Bash, short for "Bourne Again SHell", is by far the most common shell on Linux systems, and it is what every example in this handbook is written for.

A bash script is nothing more mysterious than a plain text file containing a list of commands, written in the order they should run. Instead of typing those commands into the terminal one at a time, you save them into a file, and the bash interpreter (the program that actually understands and carries out bash commands) reads that file from top to bottom and runs everything in it automatically, exactly as if you had typed each line yourself. This is what makes scripting so useful: any task you would otherwise repeat by hand, command after command, can be written down once and then run again and again with a single command.

This handbook is built around a set of real example scripts, deliberately arranged from the simplest possible programs — reading two numbers and adding them together — up through gradually more advanced ideas, ending with menu-driven tools, scripts that watch over a whole system, and scripts that fetch information from the internet. Every new idea is explained in plain language the first time it is actually needed, using an example to make it concrete, rather than all at once up front — the aim is to build up your understanding one solid step at a time, the same way the scripts themselves build in complexity. Each example shows the complete script, a plain-language explanation of what its important lines are doing and why, and the output exactly as it would appear in a real terminal.

One last practical note: every script here can be run in either of two ways, and both are used interchangeably throughout this handbook. The first is typing bash script.sh, which simply tells bash to run the file directly. The second is running chmod +x script.sh once (chmod +x marks the file as "executable", meaning the system will allow it to be run as a program in its own right) and from then on running it with ./script.sh, where the ./ just means "look for this file right here, in the current folder".

1. Getting Started: the Shebang, echo, read, and Variables

A bash script is really just a plain text file full of the same commands you could type yourself into a terminal, one after another. The computer reads the file from top to bottom and runs each line in order, exactly as if a very fast, very obedient typist had entered them for you.

Every script in this handbook starts with a special first line called the shebang — something like #!/usr/bin/env bash or #!/bin/bash. A shebang always begins with the two characters #! followed by the path to the program that should run the rest of the file. Without it, the computer would not know whether to treat the file as a bash script, a Python script, or something else entirely. #!/usr/bin/env bash is generally preferred over #!/bin/bash because it asks the system to search for bash wherever it happens to be installed (using something called the PATH, a list of folders the system checks for programs), instead of assuming it always lives in exactly one fixed folder.

echo is simply the command that prints text to the screen. Normally, after echo prints its message it also moves the cursor down to a new line, the same way pressing Enter would. Adding the -n flag (echo -n "...") tells it to skip that new line, which is useful for printing a question mark or prompt and then letting the user's answer appear right next to it instead of on its own separate line. printf is another way to print text that behaves similarly, and is used in a few scripts below for the same purpose.

read is the command that pauses a script and waits for the person running it to type something and press Enter. Whatever they type is saved into a variable — think of a variable as a small labelled box that holds a piece of information so the script can use it later. read -r a means "wait for input, then put it in the box labelled a"; the -r

flag simply tells bash not to treat a backslash character in the typed text as anything special, which is a small safety habit worth keeping.

Creating a variable in bash is done by writing its name, an equals sign, and its value, with no spaces at all around the equals sign — for example a=5. To read a variable's value back out, its name is written with a dollar sign in front of it, like $a. Wrapping it in double quotes, as in "$a", is considered the safer habit, because it keeps the value together as one piece even if it happens to contain spaces.

addition.sh

The simplest possible script: prompt for two numbers, add them, and print the result.

Script:

```
#!/usr/bin/env bash

echo -n 'Enter the First Number: '
read -r a
echo -n 'Enter the Second Number: '
read -r b
echo "$a + $b = $((a+b))"
```

How it works:

- echo -n prints the prompt text without moving to a new line, so the number the user types appears right after the colon.
- read -r a and read -r b capture the two numbers typed by the user into the variables a and b.
- $((a+b)) is arithmetic expansion (covered in the next section) — it evaluates the sum of the two variables as integers.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./addition.sh
Enter the First Number: 12
Enter the Second Number: 8
12 + 8 = 20
```

2. Arithmetic Expansion: $((...))

This is an important quirk of bash worth understanding early: bash treats everything as plain text (a string of characters) by default, even something that looks like a number. If you tried to write $a + $b on its own, bash would not add the two numbers — it would try to run a command literally named after whatever a happens to equal. To actually do maths, bash needs to be told, explicitly, "treat this as a number, not as text."

That is exactly what double parentheses do: wrapping an expression in $((...)) — called arithmetic expansion — tells bash "work this out as a sum, not as text, and give me back the number you get." So $((a + b)) evaluates the sum of a and b and substitutes the resulting number into the script at that point. Inside the parentheses, variable names can be written either with or without their leading dollar sign (both a and $a work), which is a small convenience specific to this construct.

The everyday arithmetic symbols work the way you would expect inside $((...)): + for addition, - for subtraction, * for multiplication, and / for division. One easy-to-forget detail is that bash's arithmetic only understands whole numbers (integers) — it has no idea how to represent a fraction. So when two integers do not divide evenly, the

remainder is simply thrown away rather than turning into a decimal: $((7/2)) gives 3, not 3.5. Later on, section 6 shows how a script can bring in an outside tool (bc) whenever it genuinely needs decimal answers.

subtraction.sh

Script:

```
#!/bin/bash
echo -n 'Enter the First Number: '
read a
echo -n "Enter the Second Number: "
read b
echo "$a - $b = $((a - b))"
```

How it works:

- printf and echo -n are used interchangeably across these scripts to print a prompt without a newline.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./subtraction.sh
Enter the First Number: 20
Enter the Second Number: 7
20 - 7 = 13
```

multiplication.sh

Script:

```
#!/usr/bin/env bash
echo -n "Enter the First Number: "
read a
echo -n "Enter the Second Number: "
read b
echo "$a * $b = $((a * b))"
```

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./multiplication.sh
Enter the First Number: 6
Enter the Second Number: 7
6 * 7 = 42
```

division.sh

Script:

```
#!/bin/bash
echo -n 'Enter the First Number: '
read a
echo -n "Enter the Second Number: "
read b
echo "$a / $b = $((a / b))"
```

How it works:

- Because $((...)) performs integer arithmetic, 20 / 4 gives an exact 5, but a division such as 7 / 2 would give 3, with the remainder silently dropped.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./division.sh
Enter the First Number: 20
Enter the Second Number: 4
20 / 4 = 5
```

3. Conditional Statements: if / else and Test Operators

So far every script has run the same commands every single time, in a straight line from top to bottom. Real programs usually need to make decisions — do one thing if something is true, and a different thing otherwise. That decision-making is called a conditional statement, and the most basic one in bash is if.

The shape of an if statement is: if condition; then commands to run if true; else commands to run otherwise; fi. Bash checks the condition, and if it turns out to be true, it runs everything between then and else; if the condition is false, it skips straight to the commands after else instead. The else part is optional — a script can have a plain if with no else at all if there is nothing special to do in the false case. Every if block must be closed with the word fi, which is simply "if" spelled backwards, so bash knows exactly where the conditional ends.

So how does bash actually check a condition? The most common way is the square-bracket test, written [condition]. This is really just another command in disguise (it is secretly a shorthand for a command literally called test), and it reports back whether the condition inside it is true or false. For comparing numbers specifically, bash does not use the symbols you might expect from maths class; instead it uses short two-letter codes: -eq means "equal to", -ne means "not equal to", -gt means "greater than", -lt means "less than", and -ge and -le mean "greater than or equal to" and "less than or equal to". A condition looks like [$a -eq $b], and it is important to always leave a space just inside each square bracket and around every operator, or bash will not understand the line at all.

One more small but very useful piece of arithmetic appears in this section's example: the % symbol inside $((...)) is the modulo operator, meaning "the remainder left over after dividing." For instance, $((7 % 2)) is 1, because 7 divided by 2 leaves a remainder of 1. This turns out to be exactly what is needed to tell whether a number is even or odd: any number that leaves a remainder of 0 when divided by 2 is even, and anything else is odd.

evenodd.sh

Script:

```
#!/usr/bin/env bash
echo -n "Enter The Number: "
read -r n
if [ $((n % 2)) -eq 0 ]; then
    echo "is a Even Number"
else
    echo "is a Odd Number"
fi
```

How it works:

- $((n % 2)) computes the remainder of n divided by 2.
- -eq 0 checks whether that remainder is exactly 0; if so the number is even, otherwise the else branch reports it as odd.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./evenodd.sh
Enter The Number: 7
is a Odd Number
```

4. while Loops

Sometimes a script needs to repeat the same block of commands over and over — this is called looping, and it is one of the most powerful ideas in programming, because it lets a short script do a large amount of repetitive work without the person writing it having to type the same line a hundred times.

The while loop is the simplest kind of loop bash offers. Its shape is: while condition; do commands; done. Before every single repetition, bash checks the condition; if it is still true, it runs the commands one more time and then checks again; the moment the condition becomes false, the loop stops and the script moves on to whatever comes after done. This makes while the natural choice whenever you do not know in advance exactly how many times something needs to repeat — instead, the loop keeps going for as long as some condition (often a counter, or a value the user typed in) keeps allowing it.

A very common pattern inside a while loop is a counter variable that changes a little on every pass, eventually making the condition false so the loop knows when to stop. Without something inside the loop that eventually makes the condition false, the loop would repeat forever — this is called an infinite loop, and section 18 later in this handbook deals with loops that are made infinite on purpose.

factorial.sh

Computes n! (n factorial) by repeatedly multiplying and counting down to zero.

Script:

```
#!/usr/bin/env bash
echo -n "Enter The Number: "
read -r a
fact=1
while [ "$a" -ne 0 ]; do
    fact=$((fact * a))
    a=$((a - 1))
done
echo $fact
```

How it works:

- fact starts at 1 (the multiplicative identity) and accumulates the running product.
- Each pass through the loop multiplies fact by the current value of a, then decreases a by 1.
- The loop stops as soon as a reaches 0 (-ne 0 becomes false), leaving fact holding the final factorial.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./factorial.sh
Enter The Number: 5
120
```

table.sh

Prints the multiplication table of a number, from ×2 up to ×9.

Script:

```
#!/usr/bin/env bash
echo -n "Enter The Number upto which you want to Print Table: "
read -r n
i=1
while [ $i -ne 10 ]; do
    i=$((i + 1))
    table=$((i * n))
    echo "$table"
done
```

How it works:

- The counter i begins at 1 and is incremented before it is used, so the first value printed is n × 2, and the loop keeps going until i reaches 10, printing n × 3 through n × 10 as well.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./table.sh
Enter The Number upto which you want to Print Table: 5
10
15
20
25
30
35
40
45
50
```

5. Digit Extraction with while Loops

Bash has no built-in way to look at "the third digit" of a number the way you might look at the third letter of a word — as far as bash's arithmetic is concerned, a number is just one single value, not a list of separate digits. So whenever a script genuinely needs to examine a number one digit at a time, it has to pull the digits apart itself using plain arithmetic, inside a loop.

The trick relies on two operations working together. First, modulo 10 (n % 10, using the remainder operator introduced in the previous section) always gives back the last digit of a number — for example 153 % 10 is 3. Second, integer division by 10 (n / 10) chops that last digit off the end and shifts everything else down — 153 / 10 becomes 15 (remember, from section 2, that leftover fractions are simply discarded). Repeating both steps inside a while loop, and stopping once the number has been whittled all the way down to zero, is enough to process every digit of a number, one at a time, from the last digit to the first.

armstrong.sh

An Armstrong number is one whose digits, each raised to the power of the digit count, sum back to the original number (for a 3-digit number: $1^3 + 5^3 + 3^3 = 153$).

Script:

```
#!/usr/bin/env bash
```

```
echo -n "Enter A Number: "
read -r n
arm=0
temp=$n
while [ "$n" -ne 0 ]; do
    r=$((n % 10))
    arm=$((arm + r * r * r))
    n=$((n / 10))
done
echo $arm
if [ $arm -eq "$temp" ]; then
    echo "Armstrong"
else
    echo "Not Armstrong"
fi
```

How it works:

- temp keeps a copy of the original number, because n itself is destroyed digit-by-digit inside the loop.
- r=$((n % 10)) peels off the last digit; arm accumulates its cube.
- n=$((n / 10)) drops the digit that was just processed, using integer division.
- After the loop, arm (the sum of cubes) is compared against the original number temp.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./armstrong.sh
Enter A Number: 153
153
Armstrong
```

prime.sh

Counts how many numbers from 1 up to n divide n evenly; a prime number has exactly two such divisors (1 and itself).

Script:

```
#!/usr/bin/env bash
echo -n "Enter Any Number: "
read -r n
i=1; c=1
while [ $i -le "$n" ]; do
    i=$((i + 1))
    r=$((n % i))
    [ $r -eq 0 ] && c=$((c + 1))
done

if [ $c -eq 2 ]; then
    echo "Prime"
else
    echo "Not Prime"
fi
```

How it works:

- c starts at 1 to pre-count the divisor 1, then i walks from 2 up to n, testing whether each value divides n with no remainder.

- [$r -eq 0] && c=$((c + 1)) is a short-circuit AND: the increment only runs when the remainder test on its left succeeds.
- If exactly two divisors were found by the end of the loop, the number is prime.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./prime.sh
Enter Any Number: 17
Prime
```

6. case Statements

When a script needs to choose between several different options based on one value — for example, a menu where the user types 1, 2, 3, or 4 — writing a long chain of if, then elif ("else if"), then another elif, and so on, quickly becomes messy and hard to read. bash offers a tidier tool for exactly this situation: the case statement.

A case statement takes one variable and compares it against a list of possible patterns, one after another, running only the block that matches. Its shape is: case $variable in pattern1) commands ;; pattern2) commands ;; ... esac. Each pattern ends with a closing parenthesis, each block of commands ends with a double semicolon ;; (which tells bash "stop here, do not look at any more patterns"), and the whole statement is closed with esac — simply the word "case" spelled backwards, following the same backwards-closing-word habit as if/fi.

Two small helper programs appear in this section's examples for doing maths that plain bash cannot do on its own. bc is a separate calculator program, run from the command line, that understands decimal numbers — something bash's own $((...)) cannot do, since it only works with whole numbers (see section 2). A script "feeds" a sum into bc using a pipe (written as the | symbol, which sends the output of whatever is on its left as input to whatever is on its right): echo "$a / $b" | bc prints the expression as text and then hands that text over to bc to actually work out. expr is an older, similar command used in a couple of the examples below to add, subtract, multiply, and divide.

calculator.sh

Script:

```
#!/bin/bash

#This is a simple script to calculate two numbers.

echo "Enter two numbers: "
read a
read b

echo "Enter your choice: "
echo "1. Addition"
echo "2. Substraction"
echo "3. Multiplication"
echo "4. Division"
read ch

case $ch in
```

```
   1) res=`echo $a + $b | bc`
   ;;
   2) res=`echo $a - $b | bc`
   ;;
   3) res=`echo $a \* $b | bc`
   ;;
   4) res=`echo "scale=2; $a / $b" | bc`
   ;;
esac

echo "Result: $res"
```

How it works:

- Backticks \`command\` (equivalent to the more modern $(command)) run a command and substitute its output — here, the result that bc prints back is captured into res.
- scale=2 tells bc to keep two decimal places, which matters for division since it is not restricted to whole numbers.
- Each case ... ;; branch corresponds to one menu choice; ;; marks the end of a branch so execution does not fall through to the next one.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./calculator.sh
Enter two numbers:
15
4
Enter your choice:
1. Addition
2. Substraction
3. Multiplication
4. Division
4
Result: 3.75
```

simplecalc.sh

A calculator that keeps running in a loop, asking after every calculation whether the user wants to continue.

Script:

```
#! /bin/bash
clear
sum=0
i="y"

echo " Enter one no."
read n1
echo "Enter second no."
read n2
while [ $i = "y" ]; do
    echo "1.Addition"
    echo "2.Subtraction"
    echo "3.Multiplication"
    echo "4.Division"
```

```
    echo "Enter your choice"
    read ch
    case $ch in
    1)
        sum=$(expr $n1 + $n2)
        echo "Sum ="$sum
        ;;
    2)
        sum=$(expr $n1 - $n2)
        echo "Sub = "$sum
        ;;
    3)
        sum=$(expr $n1 \* $n2)
        echo "Mul = "$sum
        ;;
    4)
        sum=$(expr $n1 / $n2)
        echo "Div = "$sum
        ;;
    *) echo "Invalid choice" ;;
    esac
    echo "Do u want to continue (y/n)) ?"
    read i
    if [ $i != "y" ]; then
        exit
    fi
done
```

How it works:

- clear wipes the terminal screen before the script starts.
- The outer while [$i = "y"] loop repeats the whole menu for as long as the user answers y to "continue?".
- The *) branch at the end of the case is a catch-all pattern that matches anything not matched above, used here to report an invalid menu choice.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./simplecalc.sh
 Enter one no.
10
Enter second no.
3
1.Addition
2.Subtraction
3.Multiplication
4.Division
Enter your choice
1
Sum =13
Do u want to continue (y/n)) ?
n
```

7. String Manipulation with tr, and Redirection

A string, in programming, simply means a piece of text — anything from a single letter to a whole sentence. Bash comes with several small tools for changing text in useful ways, and one of the most common is tr, short for "translate". tr looks at every character it is given and swaps certain characters for others. The pattern tr '[:upper:]' '[:lower:]' means "wherever you see an uppercase letter, swap it for the matching lowercase letter" — [:upper:] and [:lower:] are simply tr's built-in names for "all the uppercase letters" and "all the lowercase letters".

tr does not read a variable directly; it only reads whatever text is fed into it as input. There are two common ways to feed it that text. The first is a pipe, written |, which takes the output printed by one command and sends it straight in as input to the next command — echo "$i" | tr '[:upper:]' '[:lower:]' means "print $i, then hand what was just printed over to tr". The second is redirecting a file straight into a command using the < symbol, as in tr ... < file, which means "read the contents of this file and use it as input", without needing an echo at all.

Beyond the pipe and < that were just mentioned, bash has two more redirection symbols worth knowing, because they control where a command's output ends up. > sends a command's output into a file, completely replacing whatever was in that file before. >> also sends output into a file, but appends it onto the end instead, leaving anything already there untouched — this is the safer choice whenever a script is meant to keep adding to a file over time, such as a log.

lowercase.sh

Script:

```
#!/usr/bin/env bash
echo -n "Enter String Uppercase: "
read -r i
echo "$i" | tr '[:upper:]' '[:lower:]'
```

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./lowercase.sh
Enter String Uppercase: HELLO WORLD
hello world
```

convertlowercase.sh

Reads an entire file and appends a lowercased copy of its contents to small.txt.

Script:

```
#!/usr/bin/env bash

echo -n "Enter File Name: "
read -r file

if [ ! -f "$file" ]; then
    echo "Filename $file does not exists"
    exit 1
fi

tr '[:upper:]' '[:lower:]' < "$file" >> small.txt
```

How it works:

- [! -f "$file"] uses the file test operator -f ("is this a regular file?") together with ! (logical NOT) to check the file does not exist before continuing — file test operators are covered fully in section 10.
- < "$file" streams the file's contents into tr, and >> small.txt appends the lowercased result to small.txt without erasing anything already in it.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./convertlowercase.sh
Enter File Name: sample.txt
```

```
isha@Ubuntu:~/Desktop$ cat small.txt
sample text for testing
```

8. Command-Line Arguments: $1, $2, and $#

Every script so far has asked the user for input while it runs, using read. There is another, often quicker way to give a script information: typing extra words right after the script's name when you run it, like ./dec2hex.sh 255. These extra words are called command-line arguments, and bash automatically makes each one available inside the script through a set of ready-made variables, with no read needed at all.

Inside the script, $1 refers to the first word typed after the script's name, $2 to the second, $3 to the third, and so on. $# is a count of how many arguments were given in total, which is handy for a script to check whether it received enough information before continuing. $@ stands for "all of the arguments, kept as separate items", which matters when a script needs to loop through each one individually (this is used in section 9). Finally, $0 is a little different from the rest — it always holds the name of the script file itself, which is why usage/help messages often print $0 to remind the user what the script is called.

dec2hex.sh

Script:

```
#!/usr/bin/env bash
printf "0x%x\n" "$1"
```

How it works:

- printf's %x format specifier converts a number to lowercase hexadecimal; 0x is prefixed manually to make the result look like a hex literal.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./dec2hex.sh 255
0xff
```

hextodec.sh

Script:

```
#!/usr/bin/env bash
printf "%d\n " "$1"
```

How it works:

- printf's %d specifier accepts a 0x-prefixed hex string and prints its decimal value.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./hextodec.sh 0xff
255
```

rang-random.sh

Script:

```
#!/usr/bin/env bash
shuf -i "$1-$2" -n 1
```

How it works:

- shuf -i low-high -n 1 picks one random integer from the inclusive range low–high; $1 and $2 supply the two bounds from the command line.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./rang-random.sh 1 100
57
```

9. C-style for Loops and Arrays

Section 4 introduced the while loop, which is great when you do not know ahead of time how many times you will need to repeat something. But very often you do know exactly how many repetitions you want — "count from 0 up to 32", for instance — and for situations like that, bash offers a second, more compact kind of loop borrowed from the C programming language: for ((initializer; condition; increment)); do commands; done.

The three parts inside the double parentheses each do a specific job, run in this order every time round the loop: the initializer sets up a counter variable once, right at the very start (for example i = 32); the condition is checked before every repetition, just like in a while loop, and the loop keeps going for as long as it stays true; and the increment runs at the end of every repetition, usually changing the counter by one step (i-- lowers it by 1, i++ raises it by 1). Writing all three in one line keeps loops that simply count up or down short and easy to read.

This section also introduces the array, which is a single variable that can hold a whole list of values instead of just one. Think of it as a row of numbered boxes sharing one label, rather than a single box. New values are added onto the end of an array with name+=(value), and the whole list can be read back out, one value at a time, using ${name[@]} — this expands into every item currently stored in the array, which is exactly what lets a for loop walk through an array's contents one by one.

decimal2binary.sh

Converts one or more decimal numbers, given as command-line arguments, into their binary representation.

Script:

```
#!/usr/bin/env bash

for ((i = 32; i >= 0; i--)); do
    r=$((2 ** i))
```

```
    Probablity+=($r)
done

[[ $# -eq 0 ]] && {
    echo -e "Usage \n \t $0 numbers"
    exit 1
}

echo -en "Decimal\t\tBinary\n"
for input_int; do
    s=0
    test ${#input_int} -gt 11 && {
        echo "Support Upto 10 Digit number :: skiping \"$input_int\""
        continue
    }

    printf "%-10s\t" "$input_int"

    for n in ${Probablity[@]}; do
        if [[ $input_int -lt $n ]]; then
            [[ $s == 1 ]] && printf "%d" 0
        else
            echo -n 1
            s=1
            input_int=$((input_int - n))
        fi
    done
    echo -e
done
```

How it works:

- The first for loop builds the Probablity array containing every power of two from 2^{32} down to 2^{0} — these are the binary place values.
- for input_int; do ... done (short for for input_int in "$@"; do) loops once per command-line argument.
- For each number, the inner for n in ${Probablity[@]} loop walks the place values from largest to smallest: if the remaining value is at least as big as the place value, a 1 is printed and that amount is subtracted; otherwise a 0 is printed (once the first 1 has been seen, via the s flag, so there are no leading zeros).

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./decimal2binary.sh 10 25
Decimal             Binary
10          1010
25          11001
```

10. File Test Operators

Section 3 showed how the square-bracket test [ ] can compare two numbers. The very same square brackets can also ask true-or-false questions about files and folders on disk, using a set of file test operators — short, single-letter flags placed just before the file's name.

The most commonly used ones are: -e, which simply asks "does anything at all exist at this path?"; -f, which asks the more specific question "does this path exist, and is it an ordinary file (as opposed to, say, a folder)?"; and -d, which asks "is this path a directory (a folder)?". Three more operators check what the current user is allowed to do with a path: -r asks whether it can be read, -w asks whether it can be written to (changed), and -x asks whether it can be executed (run as a program) — or, in the case of a folder, whether its contents can be looked into at all. A condition such as [-f "$file"] therefore reads almost like plain English: "is $file a regular file?"

test-file.sh

Script:

```
#!/bin/bash
echo "Hey what's the File/Directory name (using the absolute path)?"
read FILE

if [ -e "$FILE" ]; then
    if [ -f "$FILE" ]; then
        echo "$FILE is a regular file."
    fi
    if [ -d "$FILE" ]; then
        echo "$FILE is a directory."
    fi
    if [ -r "$FILE" ]; then
        echo "$FILE is readable."
    fi
    if [ -w "$FILE" ]; then
        echo "$FILE is writable."
    fi
    if [ -x "$FILE" ]; then
        echo "$FILE is executable/searchable."
    fi
else
    echo "$FILE does not exist"
    exit 1
fi
exit
```

How it works:

- Each -f/-d/-r/-w/-x check is independent, so a file can report as both readable and writable at once — the script simply tests, and prints, every property that applies.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./test-file.sh
Hey what's the File/Directory name (using the absolute path)?
/home/isha/Desktop/sample.txt
/home/isha/Desktop/sample.txt is a regular file.
/home/isha/Desktop/sample.txt is readable.
/home/isha/Desktop/sample.txt is writable.
```

11. Positional Parameters and for-in Loops

Besides the C-style for loop from section 9, bash also has a simpler for-in loop, written for i in list; do commands; done, which walks through a ready-made list of items one at a time, putting each item into the variable i (or

whatever name is chosen) in turn. $1, $2, $3, and so on, from section 8, are together known as a script's positional parameters, and there is a shorthand specifically for looping over all of them: for i; do ... done, with no in list part at all, is understood by bash to mean exactly the same thing as for i in "$@"; do ... done.

This section's example combines that shorthand with a neat trick for turning a wildcard into a list of positional parameters in the first place. The asterisk * on its own is a wildcard, sometimes called a glob, meaning "every file and folder in the current directory"; bash automatically expands it into that full list of names before running a command. The builtin command set -- * takes advantage of this by expanding * and then loading the resulting list of names straight into the positional parameters ($1, $2, and so on) — as if the user had typed every filename directly on the command line — which is exactly what then lets the for i; do ... done shorthand loop over them.

list-dir.sh

Script:

```
#!/usr/bin/env bash

set -- *

for i; do
    echo "$i"
done
```

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./list-dir.sh
backup
bash-scripting
file.txt
file.zip
notes.txt
```

12. Command Substitution and Everyday Utilities

Not everything a script needs is built into bash itself. Linux comes with a large toolbox of small, separate programs (often called utilities or commands) that do one job well, and scripts constantly reach for them to get real work done — this section introduces two very common ones.

wc stands for "word count", and its -l flag tells it to count lines instead of words. Run it against a single file and it prints that file's line count; run it against several files at once (as ./* does, expanding to every file in the current folder) and it prints a count for each one, followed automatically by a combined total on the last line.

du stands for "disk usage" and reports how much storage space a file or folder takes up. Its -s flag means "summarize" — give one total figure for a whole folder, rather than listing every file inside it separately — and its -h flag means "human-readable", printing sizes using familiar units like K (kilobytes) or M (megabytes) instead of a raw, hard-to-read number of bytes.

count-lines.sh

Script:

```
#!/usr/bin/env bash
```

```
wc -l ./*
```

How it works:

- ./* expands to every entry in the current directory; wc -l prints a line count for each one and a combined total on the last line.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./count-lines.sh
   6 ./addition.sh
  16 ./armstrong.sh
  21 ./calculator.sh
   8 ./convertlowercase.sh
 ...
  51 total
```

directorysize.sh

Script:

```
#!/bin/bash

echo -n "Enter your directory: "
read -r x
du -sh "$x"
```

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./directorysize.sh
Enter your directory: /home/isha/bash-scripting
96K    /home/isha/bash-scripting
```

13. Interactive Prompts and String Concatenation

Joining two pieces of text together into one is called concatenation. Some programming languages need a special symbol for this (often a plus sign), but bash does not — it is done simply by placing things next to each other inside the same double-quoted string. Writing "$a $b" takes whatever is stored in a, then a literal space (because that is what was typed between them), then whatever is stored in b, and glues all three pieces into a single combined string. Any ordinary text can be mixed in the same way, as in "Hello, $a!", which combines the fixed word "Hello," with whatever value a currently holds.

interactive.sh

Script:

```
#! /bin/bash
echo "Hey what's Your First Name?"
read a
echo "welcome Mr./Mrs. $a, would you like to tell us, Your Last Name"
read b
echo "Thanks Mr./Mrs. $a $b for telling us your name"
echo "********************"
echo "Mr./Mrs. $b, it's time to say you good bye"
```

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./interactive.sh
Hey what's Your First Name?
John
welcome Mr./Mrs. John, would you like to tell us, Your Last Name
Doe
Thanks Mr./Mrs. John Doe for telling us your name
******************
Mr./Mrs. Doe, it's time to say you good bye
```

14. Building Paths and Changing Directories in Scripts

Every running program, including a bash script, always has one particular folder it considers its "current location" at any given moment — this is called the current working directory. cd (short for "change directory") moves that current location somewhere else, exactly the way typing cd at a normal terminal prompt does. $PWD is a ready-made variable (short for "print working directory") that always holds the full path of wherever the script currently is, and running the separate pwd command prints that same information.

A path can be written in two styles. An absolute path always starts from the very top of the filesystem (such as /home/isha/Desktop) and unambiguously points to one exact location no matter where the script currently is. A relative path, by contrast, is written starting from wherever the script currently happens to be — .. means "the folder one level up", and it can be chained together, so ../../ means "two levels up". The example below builds a relative path like this piece by piece, using the string concatenation from the previous section, by repeatedly gluing another ../ onto the front.

up.sh

Moves up a given number of directory levels and then drops into an interactive shell there.

Script:

```
if [ -z "$1" ]; then
    echo "Usage: $0 <LEVEL>"
    exit 1
fi

LEVEL=$1
CDIR="."

for ((i = 0; i < LEVEL; i++)); do
    echo "Level $i"
    CDIR="../$CDIR"
done

cd "$CDIR" || { echo "Failed to change directory"; exit 1; }

echo "You are in: $PWD"

$SHELL
```

How it works:

- [-z "$1"] checks whether the first argument is an empty string — -z means "zero length" — so the script can print a usage message if no level was supplied.
- The for loop builds up CDIR by prepending ../ once per level requested, turning LEVEL=2 into ../../.
- cd "$CDIR" || { ...; } uses || ("or") so the block in braces only runs if cd fails.
- $SHELL starts a fresh interactive shell in the new location, so the user stays there instead of the script simply exiting back to the old directory.

Terminal output:

```
isha@Ubuntu:~/Desktop/a/b/c$ bash up.sh 2
Level 0
Level 1
You are in: /home/isha/Desktop/a
```

15. Custom Functions and Version Sorting

As scripts grow longer, the same few lines of commands often need to run more than once, sometimes with slightly different values each time. Rather than copying and pasting those lines everywhere they are needed (which makes a script long, and a nightmare to fix if something later needs to change), bash lets a block of commands be wrapped up under one name, called a function, and then simply run by that name whenever it is needed — the same way a built-in command like echo is run just by typing its name.

A function is defined either as function name() { commands; } or the shorter name() { commands; }, and once defined, typing its name anywhere later in the script runs everything inside the curly braces, as if those lines had been typed out in full at that spot. This is sometimes described as "packaging up" a piece of logic so it can be reused cleanly.

This section's example also uses sort -V, which performs what is called a version sort. Ordinary alphabetical sorting would wrongly place "2.10" before "2.9", because it compares text character by character rather than understanding numbers — to plain text sorting, "1" simply looks smaller than "9", regardless of what comes after it. sort -V is specifically aware of version-number-style text (made of numbers separated by dots) and sorts it the way a person naturally would, comparing each number-separated part on its own numeric merit.

versioncompare.sh

Script:

```
#!/bin/bash

VERSION=$1
VERSION2=$2

function version_gt() { test "$(echo "$@" | tr " " "\n" | sort -V | head -n 1)" != "$1"; }
function version_le() { test "$(echo "$@" | tr " " "\n" | sort -V | head -n 1)" == "$1"; }
function version_lt() { test "$(echo "$@" | tr " " "\n" | sort -rV | head -n 1)" != "$1"; }
function version_ge() { test "$(echo "$@" | tr " " "\n" | sort -rV | head -n 1)" == "$1"; }

if version_gt $VERSION $VERSION2; then
    echo "$VERSION is greater than $VERSION2"
fi

if version_le $VERSION $VERSION2; then
    echo "$VERSION is less than or equal to $VERSION2"
```

```
fi

if version_lt $VERSION $VERSION2; then
    echo "$VERSION is less than $VERSION2"
fi

if version_ge $VERSION $VERSION2; then
    echo "$VERSION is greater than or equal to $VERSION2"
fi
```

How it works:

- Each helper function puts both version strings on separate lines, sorts them (forward or reverse, version-aware), and checks whether the first version ends up on top — that comparison is enough to derive greater-than, less-than, and their inclusive variants.
- Only the comparisons that are actually true print a line, so normally two of the four if blocks produce output.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./versioncompare.sh 2.5.1 2.4.9
2.5.1 is greater than 2.4.9
2.5.1 is greater than or equal to 2.4.9
```

16. Reading a File Line by Line

A very frequent task in scripting is going through a text file and doing something with each line of it in turn — checking it, reformatting it, counting it, and so on. Bash has a standard, well-known pattern (sometimes called an idiom, meaning "a commonly recognised way of writing something") for exactly this: while read -r LINE; do commands; done < file.

Reading this pattern piece by piece: < file, right at the very end, redirects the whole file so that it becomes the input for the loop, rather than waiting on the keyboard the way read normally would. Then, on every single pass through the while loop, read -r LINE pulls off just the next line of that input and stores it in the variable LINE, and the loop's condition stays true for as long as there is still another line left to read — the moment the file runs out of lines, read fails, the condition becomes false, and the loop stops automatically.

while-read.sh

Script:

```
#!/bin/bash
count=0

while read -r LINE; do
    printf "%s| %s\n" "$count" "$LINE"
    count=$(expr $count + 1)
done <$1
```

How it works:

- $1 is the filename passed on the command line; done <$1 feeds that file into the while loop line by line.
- count is a running line number, incremented with expr on each pass.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./while-read.sh fruits.txt
0| apple
1| banana
2| cherry
```

17. Nested for Loops and Pattern Printing

A loop can contain another loop entirely inside it — this is called nesting, and the inner loop runs completely, start to finish, on every single pass of the outer loop. So if the outer loop runs 5 times and the inner loop runs 3 times on each of those passes, the innermost commands actually run 15 times in total. Nested loops are the standard way to work with anything that has two dimensions, such as rows and columns — which is exactly the case for a classic beginner exercise: printing a symmetrical shape, such as a diamond or triangle, made out of a printed character. One loop steps down through the rows, one at a time, while inner loops on each row control how many blank spaces come first and how many characters get printed after them.

This section also introduces a compound test, meaning a single test that actually checks more than one condition at once. Writing [condition1 -a condition2] joins the two conditions with a logical AND (-a is bash's word for "and" inside a test), so the entire expression is only true when both conditions are true at the same time — for example, checking that a number is neither too small nor too large in one line, instead of writing two separate if checks.

special-pattern.sh

Prints a diamond made of dots, sized between 5 and 9 rows.

Script:

```
#!/bin/bash

MAX_NO=0

echo -n "Enter Number between (5 to 9) : "
read MAX_NO

if ! [ $MAX_NO -ge 5 -a $MAX_NO -le 9 ]; then
    echo "WTF... I ask to enter number between 5 and 9, Try Again"
    exit 1
fi

clear

for ((i = 1; i <= MAX_NO; i++)); do
    for ((s = MAX_NO; s >= i; s--)); do
        echo -n " "
    done
    for ((j = 1; j <= i; j++)); do
        echo -n " ."
    done
    echo ""
done
for ((i = MAX_NO; i >= 1; i--)); do
    for ((s = i; s <= MAX_NO; s++)); do
```

```
        echo -n " "
    done
    for ((j = 1; j <= i; j++)); do
        echo -n " ."
    done
    echo ""
done

echo -e "\n\n\t\t\t Whenever you need help, Tecmint.com is always there"
```

How it works:

- ! [... -a ...] negates the combined range check, so the error message only fires when the number is outside 5–9.
- The first for ((i...)) block grows the diamond's top half: each row prints fewer leading spaces and more dots than the last.
- The second for ((i...)) block, counting back down, mirrors it to produce the bottom half, completing the diamond.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./special-pattern.sh
Enter Number between (5 to 9) : 5
      .
     . .
    . . .
   . . . .
  . . . . .
  . . . . .
   . . . .
    . . .
     . .
      .


                    Whenever you need help, Tecmint.com is always there
```

18. Infinite Loops, sleep, and Cursor Control

Section 4 warned that a while loop needs something inside it that will eventually make its condition false, or else it will never stop. Sometimes, though, a script deliberately wants a loop that never stops on its own — this is called an infinite loop, and it is written either as while true; do commands; done or the equivalent until false; do commands; done. true and false here are actual bash commands that always succeed or always fail respectively, so a loop built on either of them can never naturally reach its exit condition. A script written this way is expected to keep running until the person using it presses Ctrl+C, which sends the program something called an interrupt signal — essentially a message telling it to stop right now (signals, and how a script can react to them on purpose, are covered fully in section 21).

sleep N pauses everything the script is doing for N seconds before continuing — sleep .5 pauses for half a second, since sleep also accepts fractions. Without a sleep, an infinite loop like the ones in this section would run so fast that any animation inside it would be an unreadable blur; sleep is what slows things down to a pace a human can actually watch.

Finally, this section shows how to make an animation appear to update in place, on a single line, rather than scrolling new output down the screen forever. The trick is a special, invisible character called a carriage return, written \r, which can be produced with echo -en "\r..." (the -e flag tells echo to understand special characters like \r rather than printing them literally). Printing \r moves the cursor straight back to the very start of the current line without creating a new line underneath it — so whatever gets printed immediately afterwards overwrites what was already there. Repeating that inside a loop, with a short sleep between each repetition, is the whole trick behind any spinner, live countdown, or progress bar you might have seen update in place in a terminal.

Fibonacci.sh

Prints the Fibonacci sequence forever, half a second apart.

Script:

```
#!/usr/bin/env bash

x=0; y=1; i=2
while true; do
    i=$((i + 1))
    z=$((x + y))
    echo -n "$z "
    x=$y
    y=$z
    sleep .5
done
```

How it works:

- Each iteration computes the next term z as the sum of the previous two (x and y), prints it, and then shifts x and y forward by one position — the standard iterative Fibonacci pattern.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./Fibonacci.sh
1 2 3 5 8 13 21 34 ^C
```

color.sh

Prints the word LOVE, cycling through ten ANSI colour codes, one per second, always returning the cursor to the start of the line.

Script:

```
#!/usr/bin/env bash

for c in 90 31 91 32 33 34 35 95 36 97; do
    echo -en "\r \e[${c}m LOVE \e[0m "
    sleep 1
done
```

How it works:

- \e[${c}m is an ANSI escape code that switches the terminal's text colour to the number stored in c; \e[0m resets the colour back to normal immediately after the word.

- \r at the start of the string returns the cursor to column one before printing, so each new colour overwrites the previous LOVE in place rather than printing a fresh line.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./color.sh
 LOVE   (cycles through 10 colours, once per second, in place)
```

19. Menu-Driven Scripts with Here-Documents

Printing a long, multi-line block of text — such as a menu with several numbered options — is possible with a separate echo for every line, but that quickly becomes tedious to write and to read. Bash offers a neater way to do it, called a here-document (sometimes shortened to "heredoc"), written as cat << EOF followed by several ordinary lines of plain text, and finished with a line containing only EOF.

Everything typed between the two EOF markers is passed, exactly as written, straight into whatever command comes before the <<, which here is cat (a command that simply prints back whatever it is given). The word EOF is not special in itself — any word at all can be used to mark the end of the block, as long as it appears alone on its own line at the start and end, and both markers match exactly. This makes a here-document a convenient way to print a whole menu, message, or block of instructions as one readable chunk, rather than as a long chain of separate echo lines.

read-menu.sh

Script:

```
#!/usr/bin/env bash
# read-menu: a menu driven system information program
clear
cat << EOF
Please Select:

    1. Display System Information
    2. Display Disk Space
    3. Display Home Space Utilization
    0. Quit

EOF
echo -n 'Enter selection [0-3]: '
read -r sel

case $sel in
    0) echo "Program terminated.";;
    1) echo "Hostname: $HOSTNAME"; uptime;;
    2) df -h;;
    3)
        if [ "$UID" = 0 ]; then
            echo "Home Space Utilization (All Users)"
            du -sh /home/*
        else
            echo "Home Space Utilization ($USER)"
            du -sh "$HOME"
        fi
    ;;
    *)
```

```
        echo "Invalid entry." >&2
        exit 1
esac
```

How it works:

- $UID is a read-only variable always equal to the current user's numeric ID; 0 always means the superuser (root), which is why option 3 branches on it to decide whose home directories to measure.
- The final *) branch sends its message to standard error (>&2) instead of standard output, which is the conventional place to report an invalid choice or other error.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./read-menu.sh
Please Select:

    1. Display System Information
    2. Display Disk Space
    3. Display Home Space Utilization
    0. Quit

Enter selection [0-3]: 1
Hostname: Ubuntu
 09:48:01 up 1 min,  1 user,  load average: 0.00, 0.00, 0.00
```

while-menu.sh

The same idea as read-menu.sh, but wrapped in a while true loop so the menu reappears after every choice, until the user picks 0.

Script:

```
#!/bin/bash
# while-menu: a menu driven system information program
DELAY=1 # Number of seconds to display results
while true; do
    clear
    cat << EOF
        Please Select:
        1. Display System Information
        2. Display Disk Space
        3. Display Home Space Utilization
        0. Quit
EOF
    read -p "Enter selection [0-3] > "
    case "$REPLY" in
        0)
            break
            ;;
        1)
            echo "Hostname: $HOSTNAME"
            uptime
            ;;
        2)
            df -h
```

```
            ;;
        3)
            if [[ $(id -u) -eq 0 ]]; then
                echo "Home Space Utilization (All Users)"
                du -sh /home/*
            else
                echo "Home Space Utilization ($USER)"
                du -sh $HOME
            fi
            ;;
        *)
            echo "Invalid entry."
            ;;
    esac
    sleep "$DELAY"
done
echo "Program terminated."
```

How it works:

- read -p "..." prints its prompt and reads input in a single step, storing the result in the default variable $REPLY since no variable name was given.
- break immediately exits the enclosing while loop, which is how choosing 0 ends the menu.
- sleep "$DELAY" pauses briefly after each action so the result stays on screen before the menu clears and redraws.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./while-menu.sh
        Please Select:
        1. Display System Information
        2. Display Disk Space
        3. Display Home Space Utilization
        0. Quit
Enter selection [0-3] > 2
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        40G   18G   20G  48% /
(menu redraws after a 1-second pause)
```

20. Multi-Level Menus Built from Functions

Once a script grows into a proper multi-option tool, it becomes worth using functions (introduced in section 15) to keep it organised: each menu choice gets its own small, clearly-named function, so it is obvious at a glance what each part of the script is responsible for, instead of everything being crammed into one long block.

This section's example also introduces a function calling itself, which is known as recursion. After the user picks a menu option and that option's action finishes running, the script calls its own main menu function all over again — so the menu reappears and the user can pick another option, rather than the script simply reaching the end and closing. It keeps re-calling itself, in effect looping forever, until the user picks the option that runs exit and actually closes the program.

hardware_machine.sh

A larger menu-driven tool that reports on different aspects of the machine (CPU, kernel, installed software, memory, and more), calling itself again after each action so the menu keeps reappearing.

Script:

```
function menuprincipal () {
clear
TIME=1
echo "Choose an option below!

        1 - Verify desktop processor
    2 - Verify system kernel
    3 - Verify installed softwares
    4 - Operation system version
           5 - Verify desktop memory
    6 - Verify serial number
    7 - Verify system IP
    0 - Exit"
read opcao
case $opcao in
    1)
        function processador () {
            CPU_INFO=`cat /proc/cpuinfo | grep -i "^model name" | cut -d ":" -f2 | sed -n '1p'`
            echo "CPU model: $CPU_INFO"
            sleep $TIME
        }
        processador
        read -n 1 -p "<Enter> for main menu"
        menuprincipal
        ;;
    5)
        function memory () {
            MEMORY_FREE=`free -m  | grep ^Mem | tr -s ' ' | cut -d ' ' -f 4`
            echo Verifying system memory...
            echo "Memory free is: $MEMORY_FREE"
        }
        memory
        read -n 1 -p "<Enter> for main menu"
        menuprincipal
        ;;
    0)
           echo Exiting the system...
                  sleep $TIME
           exit 0
           ;;
esac
}
menuprincipal
```

How it works:

- Each numbered branch of the case defines a small function on the spot (for example processador or memory) and immediately calls it — a way of keeping each action's logic self-contained.

- cut -d ":" -f2 splits a line on the : character and keeps the second field; sed -n '1p' prints only the first matching line. Together with grep, this is a common pipeline for pulling one specific value out of a larger command's output.
- After an action finishes, the script calls menuprincipal again — the menu function calling itself — which is what redraws the main menu instead of the script exiting.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./hardware_machine.sh
Choose an option below!

        1 - Verify desktop processor
       2 - Verify system kernel
       3 - Verify installed softwares
       4 - Operation system version
              5 - Verify desktop memory
       6 - Verify serial number
       7 - Verify system IP
       0 - Exit
Chosen option: 1
CPU model:  Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz
<Enter> for main menu
```

21. Signal Handling with trap

A signal is a short message the operating system can send to a running program, usually to tell it that something has happened — the most familiar example is what happens when a person presses Ctrl+C in a terminal: this sends the running program a signal named INT (short for "interrupt"), which normally causes it to stop immediately.

By default, a script has no say in the matter and simply stops the moment it receives most signals. The trap command changes that: writing trap command signal tells bash "if this particular signal ever arrives, run this command first" — for example, trap finish EXIT means "always run the finish function whenever the script is about to end, no matter what causes that to happen." EXIT here refers to a special built-in signal-like event that bash triggers whenever a script is finishing up, for any reason at all. This is the standard, reliable way to guarantee that some cleanup step — closing a file, printing a farewell message, removing a temporary file — always happens before a script actually closes, even if the user cuts it off early with Ctrl+C.

traps.sh

Script:

```
#!/bin/bash

function finish() {
    # your cleanup here.
    echo "CTL+C pressed"
    echo "clean ..."
    sleep 1
}
trap finish EXIT

echo 'runing ...'
until false; do
```

```
    sleep 1
done
```

How it works:

- until false; do ... done is another way of writing an infinite loop, since the condition false is, by definition, never true and until keeps going while its condition is false.
- trap finish EXIT means the finish function runs no matter how the script's execution ends — including when the user interrupts it with Ctrl+C.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./traps.sh
runing ...
^C
CTL+C pressed
clean ...
```

22. Process Information and Simple File Logging

Every program that is currently running on a computer, including the terminal itself and the script currently executing, is called a process. The ps command lists the processes that are currently active, which is useful for checking what a system is busy doing at any given moment.

$USER is one of a handful of variables that bash sets up automatically, without a script needing to define it — its value is always the name of whichever person is currently logged in and running the script.

This section also introduces command substitution, written $(command), which is closely related to the arithmetic expansion from section 2 but for commands in general rather than just sums: it runs whatever command is written inside the parentheses, and then substitutes that command's printed output directly into the surrounding text, as if it had been typed there by hand. $(date), for example, runs the date command and drops today's date and time straight into a string. This is often combined with the append redirection >> from section 7 to build up a simple log file over time — each run of the script adds one more line, based on a freshly-generated piece of text, without disturbing anything written by previous runs.

process.sh

Script:

```
#!/usr/bin/env bash
echo "Hello $USER"
echo "Hey i am $USER and will be telling you about the current processes"
echo "Running processes List"
ps
```

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./process.sh
Hello isha
Hey i am isha and will be telling you about the current processes
Running processes List
  PID TTY          TIME CMD
 2041 pts/0    00:00:00 bash
 2210 pts/0    00:00:00 process.sh
```

```
 2211 pts/0    00:00:00 ps
```

randomfile.sh

Script:

```
#!/usr/bin/env bash

echo "Hello $USER"
uptime >> "$(date)".txt
echo "Your File is being saved to $(pwd)"
```

How it works:

- $(date) is used directly as part of a filename, so a new, uniquely-named file is created (and appended to) every time the script runs at a different second.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./randomfile.sh
Hello isha
Your File is being saved to /home/isha/Desktop
```

23. Talking to the Network with curl

So far every example has worked entirely with information already on the computer. curl is a tool that lets a script reach out over the internet instead: given a web address (a URL), it fetches whatever is there and prints it out, in exactly the same way a web browser would, but without any of the visual formatting — just the raw underlying content. This makes curl the standard building block for any script that needs live information from the web, such as a weather forecast or the answer from an online lookup service (these online services designed to be used by other programs, rather than by a person clicking around, are often called APIs).

Many such services reply using a structured text format called JSON, which organises information into clearly labelled fields — for example a field called city holding a place name. Reading a specific value out of a JSON reply by hand would be fiddly, so scripts commonly use a companion tool called jq (a JSON processor) to do it instead. Writing jq '.city' picks out just the value stored in the city field from a JSON reply and prints only that, ignoring everything else in the response.

weather.sh

Fetches an ASCII-art weather report from the free wttr.in service, optionally for a named location.

Script:

```
#!/bin/bash
program=Weather
version=1.1
year=2018
developer=computer-geek64

case $1 in
-h | --help)
    echo "$program $version"
    echo "Copyright $year $developer. All rights reserved."
```

```
    echo
    echo "Usage: weather [options]"
    echo "Option          Long Option            Description"
    echo "-h              --help                 Show the help screen"
    echo "-l [location]   --location [location]   Specifies the location"
    ;;
-l | --location)
    curl https://wttr.in/$2
    ;;
*)
    curl https://wttr.in
    ;;
esac
```

How it works:

- -h | --help inside a case pattern matches either spelling of the flag with a single branch — the vertical bar works as an OR inside a case pattern the same way it can inside [[ ]].

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./weather.sh -l London
Weather report: London

     \   /     Clear
      .-.      14 °C
   ― (   ) ―   ↗ 11 km/h
      `-’      10 km
     /   \     0.0 mm
```

whereIP.sh

Looks up the city, region, and country for a given IP address using the free ip-api.com service.

Script:

```
#!/usr/bin/env bash

if ! [ -x "$(command -v jq)" ]; then
  echo 'Error: jq is not installed. Install via https://stedolan.github.io/jq/download/'
  exit 1
fi

if [[ $# -ne 1 ]]; then
    echo 'Provide I.P as command line parameter. Usage:  ' $0 ' 15.45.0.1 '
    exit 1
fi
link=$(echo "http://ip-api.com/json/"$1)
data=$(curl $link -s)

status=$(echo $data | jq '.status' -r)

if [[ $status == "success" ]]; then
    city=$(echo $data | jq '.city' -r)
    regionName=$(echo $data | jq '.regionName' -r)
```

```
    country=$( echo $data | jq '.country' -r)
    echo $city, $regionName in $country.
fi
```

How it works:

- command -v jq checks whether the jq program is installed and on the PATH, printing an install hint and exiting early if it is missing — a common defensive pattern before relying on an external tool.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./whereIP.sh 8.8.8.8
Mountain View, California in United States.
```

24. Batch Image Processing

convert is a command that comes from a well-known collection of image tools called ImageMagick, and it lets a script edit pictures directly from the command line — resizing them, changing their format, and a great deal more — without opening any image-editing program by hand. This turns a task that would otherwise mean clicking through dozens of files one at a time into something a short script can do automatically.

Doing the same thing to many files at once like this is often called batch processing ("a batch" simply meaning a whole group handled together). The standard way to batch-process a folder of images in bash is to combine a wildcard (introduced in section 11 — for example *.jpg, meaning "every file ending in .jpg") with a for-in loop, so the same convert command runs once per matching file, automatically, without needing to name each file individually.

thumbnail.sh

Script:

```
#!/bin/bash
for i in *.jpg; do
    convert "$i" -thumbnail 400 "thumbs/$i";
done;
```

How it works:

- -thumbnail 400 resizes each image so its longest side is 400 pixels, and the result is written into a thumbs subfolder using the same filename as the original.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./thumbnail.sh
(no output on success — check the thumbs folder)
```

```
isha@Ubuntu:~/Desktop/photos$ ls thumbs/
holiday.jpg
sunset.jpg
team.jpg
```

25. System Monitoring and Email Alerts

The scripts in this section are not meant to be run once by hand — they are designed to be run automatically and repeatedly in the background, typically by cron, which is a built-in Linux scheduler that can be told to run a given script every minute, every hour, or on any other repeating schedule. Run this way, a short script can quietly keep watch over a system around the clock, without anyone needing to remember to check on it.

mail -s "subject" address is the command these scripts use to actually send an email from the command line: -s sets the subject line, and whatever text is piped into mail (using the pipe symbol | from section 6) becomes the body of the message. This lets a script raise the alarm to a real person the moment it detects a genuine problem, rather than the warning simply sitting unnoticed in a log file.

awk is a small, separate text-processing language, mentioned here because these examples lean on it heavily to pull one particular number out of a much longer block of text that another command has printed (output arranged neatly into rows and columns like this is often called tabular output). A full explanation of awk is beyond this handbook's scope, but it is worth recognising as "a tool for slicing up structured text" whenever it appears in a one-line pipeline like the ones below.

cpu.sh

Emails an alert if CPU usage climbs above a threshold.

Script:

```
MAX=95
EMAIL=server@127.0.0.1

USE=$(grep 'cpu ' /proc/stat | awk '{usage=($2+$4)*100/($2+$4+$5)} END {print usage ""}')
if [ $USE -gt $MAX ]; then
    echo "Percent used: $USE" | mail -s "Running out of CPU power" $EMAIL
fi
```

How it works:

- /proc/stat is a virtual file the Linux kernel maintains with live CPU statistics; the awk expression turns a few of its columns into a rough percentage-used figure.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./cpu.sh
(silent — an email is only sent if usage exceeds 95%)
```

disk-space.sh

Script:

```
MAX=95
EMAIL=server@127.0.0.1
PART=sda1

USE=$(df -h | grep $PART | awk '{ print $5 }' | cut -d'%' -f1)
if [ $USE -gt $MAX ]; then
    echo "Percent used: $USE" | mail -s "Running out of disk space" $EMAIL
fi
```

How it works:

- df -h reports disk usage per partition; the pipeline filters it down to the named partition, then extracts just the use-percentage column and strips off the trailing % sign so it can be compared as a number.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./disk-space.sh
(silent — an email is only sent if usage exceeds 95%)
```

get-temperature.sh

Script:

```
#!/usr/bin/env bash

TEMP_FILE=/sys/class/thermal/thermal_zone0/temp

ORIGINAL_TEMP=$(cat $TEMP_FILE)
TEMP_C=$((ORIGINAL_TEMP/1000))
TEMP_F=$(($TEMP_C * 9/5 + 32))

echo $TEMP_F F
```

How it works:

- /sys/class/thermal/.../temp is a virtual file exposing the CPU temperature sensor in millidegrees Celsius, which is why the raw reading is divided by 1000 before the Celsius-to-Fahrenheit formula is applied.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./get-temperature.sh
113 F
```

26. System Administration and Reporting Scripts

System administration simply refers to the everyday work of keeping a computer or server running smoothly — checking on it, maintaining it, and troubleshooting problems when something goes wrong. The scripts in this section are examples of a very common administration pattern: rather than doing anything especially clever with bash syntax itself, they chain together many small, ready-made reporting utilities — uptime (how long the machine has been running), w (who is currently logged in), last (a history of recent logins), top (which processes are using the most resources), and several more — one after another, into a single script.

Run all together, one after another, these separate little reports combine into one long, comprehensive summary of a machine's health, which is exactly the sort of thing worth having on hand when troubleshooting an issue or sending a status update to a colleague. Each utility used here is an ordinary external program, not a special part of bash — the skill being demonstrated is knowing which existing tools to reach for and how to stitch their output together into one useful script.

server-health.sh

Builds a one-shot health report of a server: uptime, logged-in users, recent logins, disk and memory usage, top processes, open ports, and more.

Script:

```
#!/bin/bash
date
echo "uptime:"
uptime
echo "Currently connected:"
w
echo "--------------------"
echo "Last logins:"
last -a | head -3
echo "--------------------"
echo "Disk and memory usage:"
df -h | xargs | awk '{print "Free/total disk: " $11 " / " $9}'
free -m | xargs | awk '{print "Free/total memory: " $17 " / " $8 " MB"}'
echo "--------------------"
echo "Utilization and most expensive processes:"
top -b | head -3
echo
top -b | head -10 | tail -4
echo "--------------------"
echo "Open TCP ports:"
nmap -p -T4 127.0.0.1
echo "--------------------"
echo "Current connections:"
ss -s
echo "--------------------"
echo "processes:"
ps auxf --width=200
echo "--------------------"
echo "vmstat:"
vmstat 1 5
```

How it works:

- top -b runs top in "batch mode", printing a single static snapshot instead of a live, refreshing display, which makes its output usable inside a script.
- Each section is separated visually by a line of dashes and a descriptive echo, purely to keep the final report readable.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./server-health.sh
Thu Sep 17 09:52:01 UTC 2026
uptime:
 09:52:01 up 14 days,  3:21,  2 users,  load average: 0.15, 0.22, 0.19
Currently connected:
 09:52:01 up 14 days,  3:21,  2 users,  load average: 0.15, 0.22, 0.19
USER     TTY      LOGIN@   IDLE   JCPU   PCPU WHAT
isha     pts/0    09:10    0.00s  0.04s  0.00s w
--------------------
Disk and memory usage:
Free/total disk: 20G / 40G
Free/total memory: 3102 / 7963 MB
```

collectnetworkinfo.sh

A root-only script that gathers a machine's complete networking configuration (kernel, PCI devices, routing table, firewall rules, and more) into a dated text file, ready to be emailed to support.

Script:

```
#!/bin/bash

IP4FW=/sbin/iptables
LSPCI=/usr/bin/lspci
ROUTE=/sbin/route
NETSTAT=/bin/netstat
LSB=/usr/bin/lsb_release
DNSCLIENT="/etc/resolv.conf"
OUTPUT="network.$(date +'%d-%m-%y').info.txt"
SUPPORT_ID="your_name@service_provider.com"

chk_root() {
    local meid=$(id -u)
    if [ $meid -ne 0 ]; then
        echo "You must be root user to run this tool"
        exit 999
    fi
}

write_header() {
    echo "----------------------------------------------------" >>$OUTPUT
    echo "$@" >>$OUTPUT
    echo "----------------------------------------------------" >>$OUTPUT
}

dump_info() {
    echo "* Hostname: $(hostname)" >$OUTPUT
    echo "* Run date and time: $(date)" >>$OUTPUT

    write_header "Linux Distro"
    echo "Linux kernel: $(uname -mrs)" >>$OUTPUT
    $LSB -a >>$OUTPUT

    write_header "PCI Devices"
    ${LSPCI} -v >>$OUTPUT

    write_header "DNS Client $DNSCLIENT Configuration"
    [ -f $DNSCLIENT ] && cat $DNSCLIENT >>$OUTPUT || echo "Error $DNSCLIENT file not found."
>>$OUTPUT

    write_header "IP4 Firewall Configuration"
    $IP4FW -L -n >>$OUTPUT

    echo "The Network Configuration Info Written To $OUTPUT. Please email this file to
$SUPPORT_ID."
}

chk_root
dump_info
```

How it works:

- local meid=$(id -u) declares meid as a variable scoped only to the chk_root function, rather than leaking into the rest of the script — good practice inside any function.
- The && ... || ... pattern after the -f test is a compact if/else: if the file exists, cat it; otherwise (||), print the error line instead.
- $(date +'%d-%m-%y') formats the current date as part of the output filename, so each run's report is kept in its own dated file.

Terminal output:

```
isha@Ubuntu:~/Desktop$ sudo ./collectnetworkinfo.sh
The Network Configuration Info Written To network.17-09-26.info.txt. Please email this file to
your_name@service_provider.com.
```

27. Archiving and Encryption

tar is a command that bundles a whole folder, including everything inside it, into one single file called an archive — making a group of many files much easier to move around, send, or back up as a single unit. Its most commonly used flags are -c (create a new archive), -z (also compress the archive using a well-known compression method called gzip, to make the resulting file smaller), -v (verbose, meaning it prints the name of every file as it gets added, so you can see progress), and -f (the flag right before the archive's chosen filename).

gpg -c takes this a step further by encrypting a file, meaning it scrambles the file's contents so that it becomes unreadable to anyone without the right key. The -c flag specifically asks for symmetric encryption, which means one single secret (called a passphrase — essentially a password) is used to both lock and later unlock the file; gpg will prompt whoever runs the command to type and confirm that passphrase on the spot, and the very same passphrase will be required later to decrypt (unlock) the file again.

archive-and-encrypt.sh

Script:

```
#!/usr/bin/env bash
name=$1
path=$2
tar -czvf "$name.tar.gz" "$path"
gpg -c "$name.tar.gz"
rm -rf "$name.tar.gz"
```

How it works:

- After the encrypted .gpg file is created, the plain (unencrypted) .tar.gz archive is deleted with rm -rf, leaving only the protected copy on disk.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./archive-and-encrypt.sh backup ~/Documents
Documents/
Documents/report.docx
Documents/notes.txt
Enter passphrase:
Repeat passphrase:
```

```
(backup.tar.gz.gpg created; backup.tar.gz removed)
```

encrypt.sh

Script:

```
#!/usr/bin/env bash

echo "Welcome, I am ready to encrypt a file/folder for you"
echo "currently I have a limitation, Place me to the same folder, where a file to be encrypted
is present"
echo "Enter the Exact File Name with extension"
read -r file
gpg -c "$file"
echo "I have encrypted the file sucessfully..."
echo "Now I will be removing the original file"
rm -rf "$file"
```

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./encrypt.sh
Welcome, I am ready to encrypt a file/folder for you
currently I have a limitation, Place me to the same folder, where a file to be encrypted is
present
Enter the Exact File Name with extension
notes.txt
Enter passphrase:
I have encrypted the file sucessfully...
Now I will be removing the original file
```

28. Remote Backups with rsync

rsync is a tool built specifically for keeping two locations in sync with one another — commonly a folder on the local machine and a folder on a separate, remote server reached over the network. Its key advantage over simply copying files fresh every time is that, on every run after the first, rsync compares what is already at the destination and transfers only the parts that have actually changed since the previous backup, which makes repeated backups of the same large folder far faster than copying everything all over again.

A few flags are used together so often that they show up almost as a set phrase: -a (archive mode) tells rsync to preserve extra file details, such as permissions and timestamps, exactly as they are, rather than just copying the raw file content; -v is verbose, printing what rsync is doing as it goes; and -z compresses the data while it travels over the network, to save time on the transfer. -e "ssh" tells rsync to send everything through SSH, a secure, encrypted connection method, which matters whenever a backup is travelling over the open internet to a remote server rather than staying on one machine.

remotebackup.sh

Script:

```
#!/bin/bash

rsync -avz -e "ssh " /path/to/yourfile user@backupserver.com:/backup/
echo "backup for $(date) " | mail -s "backup complete" user@youremail.com
```

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./remotebackup.sh
sending incremental file list
yourfile

sent 1,048,704 bytes  received 35 bytes  209,747.80 bytes/sec
total size is 1,048,576  speedup is 1.00
```

29. Time-Based Scripts and Cross-Platform Checks

Not every computer works the same way underneath, even though they might all be able to run a bash script. A script meant to be shared and used on different kinds of computers sometimes needs to check, while it is running, exactly which operating system it is on, and then behave slightly differently depending on the answer — this is called a cross-platform check. uname is the command used for this: it reports back the name of the current operating system, printing "Darwin" for macOS or "Linux" for a Linux system, which lets a script pick the one notification command that will actually work — osascript on macOS, or notify-send on Linux — rather than assuming every user's computer works the same way.

This section's example also shows the standard way bash builds a countdown timer. $(date +%s) is a command substitution (see section 22) that returns the current moment in time as one single, ever-increasing number — the count of seconds that have passed since a fixed reference point back in 1970, known as a Unix timestamp. Because it is just a plain number, ordinary arithmetic (section 2) can be used on it directly: adding a number of seconds to the current timestamp calculates exactly which future moment a countdown should end at, and repeatedly checking "has that moment arrived yet?" inside a loop is all a timer really needs to do.

pomodoro.sh

A Pomodoro-technique timer: counts down a focus period, then a break period, notifying the user (differently depending on OS) at each transition, and repeats.

Script:

```
#!/bin/bash
wseconds=${1:-25}*60;
pseconds=${2:-wseconds/300}*60;

if [ "$(uname)" == "Darwin" ]; then
    while true; do
        date1=$((`date +%s` + $wseconds));
        while [ "$date1" -ge `date +%s` ]; do
            echo -ne "$(date -u -j -f %s $(($date1 - `date +%s`)) +%H:%M:%S)\r";
        done
        osascript -e 'display notification "Time to walk and rest!" with title "Break"';
        read -n1 -rsp $'Press any key to continue or Ctrl+C to exit...\n';
    done
elif [ "$(expr substr $(uname -s) 1 5)" == "Linux" ]; then
    while true; do
        date1=$((`date +%s` + $wseconds));
        while [ "$date1" -ge `date +%s` ]; do
            echo -ne "$(date -u --date @$(($date1 - `date +%s` )) +%H:%M:%S)\r";
        done
        notify-send "Break" "Time to walk and rest";
```

```
        read -n1 -rsp $'Press any key to continue or Ctrl+C to exit...\n';
    done
else
    echo -ne "Your OS is currently not supported\n";
fi
```

How it works:

- ${1:-25} means "use the first argument if one was given, otherwise default to 25" — a common bash pattern for optional parameters with sensible defaults.
- date1=$((`date +%s` + $wseconds)) computes the timestamp at which the focus period should end, and the inner while loop keeps refreshing a live countdown (via \r) until that moment arrives.
- uname distinguishes macOS ("Darwin") from Linux so the script can call the notification tool each platform actually provides.

Terminal output:

```
isha@Ubuntu:~/Desktop$ ./pomodoro.sh 25 5
00:24:58
(countdown refreshes in place every second)
00:00:00
(desktop notification: "Break — Time to walk and rest!")
Press any key to continue or Ctrl+C to exit...
```

Closing Notes

Taken together, these examples cover the core vocabulary of bash scripting: variables and I/O, arithmetic, conditionals, the two main loop forms, case statements, functions, arrays, file tests, string and text-processing tools, and the everyday external commands (curl, tar, gpg, rsync, du, df, ps and friends) that scripts lean on to get real work done. From here, the same building blocks combine, in different orders, to write almost any automation task a Linux system might need.